

Lab 1 Report: Running Synthetic Traffic

gem5 Garnet Network-on-Chip Simulation

Name: _____
Student ID: _____

gem5 v23.0.0.1

Abstract

This laboratory studies the Garnet 3.0 network model in gem5 using synthetic traffic. A 64-node, 8×8 mesh is exercised with uniform-random, single-flit control traffic. The global tick frequency is corrected to 2 GHz, statistical units are added to Garnet, and the measured packet rate, latency, and hop count are analyzed. The final run injects 3165 packets and receives 3162 packets, with a reception rate of 0.004940625 packets/node/network-cycle.

Contents

1	Experimental Setup	2
2	Task 1: Frequency Correction and Test Run	2
2.1	Global frequency change	2
2.2	Measured results	2
3	Task 2: Statistical Units and Reception Rate	3
4	Task 3: Configuration and Source-Code Analysis	3
4.1	Input parameters and defaults	3
4.2	Units of cycles, router latency, and link latency	4
4.3	Injection-rate unit	5
4.4	Network interface and router definitions	5
4.5	Packet generation, buffering, and downstream eligibility	5
5	Conclusion	5

1 Experimental Setup

The experiment uses gem5 tag v23.0.0.1, commit af72b9ba580546ac12ce05bfaac3fd53fa8699f4, built for the NULL ISA with the Garnet_standalone protocol. The relevant incremental build command is:

```
scons --ignore-style build/NULL/gem5.opt \  
  PROTOCOL=Garnet_standalone -j$(nproc)
```

SCons tracks dependencies, so changing `GarnetNetwork.cc` recompiles the affected Garnet object and relinks `gem5.opt`; it does not require deleting the build directory or recompiling every source file.

The test configuration is:

```
./build/NULL/gem5.opt \  
  configs/example/garnet_synth_traffic.py \  
  --network=garnet --num-cpus=64 --num-dirs=64 \  
  --topology=Mesh_XY --mesh-rows=8 \  
  --inj-vnet=0 --synthetic=uniform_random \  
  --sim-cycles=10000 --injectionrate=0.01
```

The 64 objects named CPUs are synthetic traffic generators rather than instruction-executing processors. `Mesh_XY` creates 64 routers arranged as eight rows by eight columns. There are also 64 directory destinations. Virtual network 0 carries one-flit control packets, and every source chooses a destination uniformly from all 64 destinations, including itself.

2 Task 1: Frequency Correction and Test Run

2.1 Global frequency change

The original configuration used:

```
m5.ticks.setGlobalFrequency("1ps")
```

This means one tick is 1 ps, or 10^{12} ticks/s. It was changed to:

```
m5.ticks.setGlobalFrequency("2GHz")
```

Source: `/root/gem5/configs/example/garnet_synth_traffic.py`

The final statistics confirm `simFreq = 2000000000 ticks/s` and `simTicks = 10000`. Therefore,

$$T_{\text{tick}} = \frac{1}{2 \times 10^9} = 0.5 \text{ ns}, \quad T_{\text{sim}} = 10000 \times 0.5 \text{ ns} = 5 \mu\text{s}. \quad (1)$$

The system clock is 1 GHz, while Ruby/Garnet runs at 2 GHz. Consequently,

$$1 \text{ system cycle} = 2 \text{ ticks} = 1 \text{ ns}, \quad 1 \text{ Ruby cycle} = 1 \text{ tick} = 0.5 \text{ ns}. \quad (2)$$

The run thus contains approximately 5000 traffic-generator cycles and 10000 Ruby network cycles.

2.2 Measured results

At an injection probability of 0.01 per node per system cycle, the expected number of generated packets is approximately

$$64 \text{ nodes} \times 5000 \text{ cycles} \times 0.01 = 3200 \text{ packets}. \quad (3)$$

Table 1: Statistics before and after correcting the global frequency.

Metric	Original 1 ps/tick	Final 2 GHz
Packets injected	5	3165
Packets received	2	3162
Average queueing latency	1000 ticks	2 ticks/packet
Average network latency	3000 ticks	13.557559 ticks/packet
Average total latency	4000 ticks	15.557559 ticks/packet
Average hops	1.500000	5.269450

The observed 3165 injections are consistent with this Bernoulli process. Only three packets remain in flight at termination. For a uniform-random 8×8 mesh, the expected Manhattan distance is

$$E[H] = 2E[|X_s - X_d|] = 2 \frac{8^2 - 1}{3 \times 8} = 5.25 \text{ hops.} \quad (4)$$

The measured 5.269450 hops differs from the theoretical value by only 0.37%, which is reasonable for 3162 received packets.

3 Task 2: Statistical Units and Reception Rate

The original Garnet statistics were emitted as (Unspecified). Unit metadata was added in `GarnetNetwork::regStats()` without changing any counter or network behavior.

Table 2: Units assigned to Garnet statistical variables.

Statistics	Unit
Injected/received packets and flits; traffic distribution	Count
Accumulated packet and flit latency	Tick
Average packet and flit latency	Tick/Count
Total hops and link activity	Count
Average hops	Count/Count
Average link utilization and VC load	Ratio

Source: `/root/gem5/src/mem/ruby/network/garnet/GarnetNetwork.cc`

The required reception rate is normalized by node count and the 10000 network cycles specified by the laboratory:

$$R_{\text{recv}} = \frac{N_{\text{received}}}{N_{\text{nodes}} N_{\text{cycles}}} = \frac{3162}{64 \times 10000} = 0.004940625 \frac{\text{packets}}{\text{node} \cdot \text{cycle}}. \quad (5)$$

This is a throughput, not a delivery ratio. The delivery ratio is instead $3162/3165 \approx 99.905\%$. The offered rate is 0.01 packets/node/system-cycle. Since one system cycle spans two Ruby cycles, the equivalent offered network rate is 0.005 packets/node/Ruby-cycle, close to the measured reception rate.

4 Task 3: Configuration and Source-Code Analysis

4.1 Input parameters and defaults

All accepted options can be displayed with:

```
./build/NULL/gem5.opt configs/example/garnet_synth_traffic.py -h
```

The test-specific options are summarized below.

Table 3: Synthetic traffic options.

Option	Default	Purpose
<code>-synthetic</code>	<code>uniform_random</code>	Destination pattern: uniform random, tornado, bit complement, bit reverse, bit rotation, neighbor, shuffle, or transpose.
<code>-injectionrate</code>	0.1	Packet-generation probability per node per traffic-generator cycle.
<code>-precision</code>	3	Decimal precision used by the random injection test.
<code>-sim-cycles</code>	1000	Simulation termination threshold; in this version it is compared directly with <code>curTick()</code> .
<code>-num-packets-max</code>	-1	Maximum packets generated by each sender; -1 disables the limit.
<code>-single-sender-id</code>	-1	Restrict injection to one sender; -1 enables every sender.
<code>-single-dest-id</code>	-1	Force one destination; -1 uses the selected traffic pattern.
<code>-inj-vnet</code>	-1	Select vnet 0, 1, or 2; -1 selects among them randomly. Vnets 0/1 use one-flit control packets and vnet 2 uses five-flit data packets.

These defaults are defined directly in `configs/example/garnet_synth_traffic.py`. General system and memory options such as `-num-cpus`, `-num-dirs`, `-sys-clock`, memory type, cache geometry, and maximum tick come from `configs/common/Options.py`. Ruby defaults, including the 2 GHz Ruby clock, come from `configs/ruby/Ruby.py`. Network defaults come from `configs/network/Network.py`: Crossbar topology, simple network, one-cycle router and link latency, 128-bit links, four VCs per vnet, weight-table routing, and a 50000-cycle deadlock threshold.

For this experiment, `Mesh_XY.py` assigns horizontal link weight 1 and vertical link weight 2. The default weight-based routing table therefore enforces dimension-ordered XY routing. Alternatively, `-routing-algorithm=1` selects the explicit XY implementation in `RoutingUnit.cc`.

4.2 Units of cycles, router latency, and link latency

A tick is the global, integer simulation time quantum. A cycle belongs to a specific clock domain. Their relationship is

$$\text{ticks per component cycle} = \frac{f_{\text{global}}}{f_{\text{component}}}. \quad (6)$$

`router-latency` is a number of Ruby/router cycles and represents router pipeline stages. An arriving flit is held in its input VC for `router-latency - 1` cycles before switch allocation. Likewise, `link-latency` is a number of network-link cycles; the link marks the flit ready at `clockEdge(link_latency)`.

Although the command-line help describes `sim-cycles` as cycles, `GarnetSyntheticTraffic::tick()` compares `curTick()` directly with `simCycles`. Its strict implementation unit in v23.0.0.1 is therefore Tick. With both global and Ruby frequencies at 2 GHz, one tick happens to equal one Ruby cycle.

4.3 Injection-rate unit

Each `GarnetSyntheticTraffic` object wakes once per system clock cycle. It draws a random integer and generates a request when the draw is below `injRate` times the precision range. Thus the injection-rate unit is packets/node/system-cycle. It is an offered probabilistic load, not a guarantee that exactly that many packets enter or leave the network.

4.4 Network interface and router definitions

The Python `SimObject` declarations and their C++ implementations are:

SimObject	Python declaration	C++ implementation
<code>GarnetNetworkInterface</code>	<code>GarnetNetwork.py</code>	<code>NetworkInterface.hh/.cc</code>
<code>GarnetRouter</code>	<code>GarnetNetwork.py</code>	<code>Router.hh/.cc</code>

The files reside under `src/mem/ruby/network/garnet/`. The configuration layer in `configs/network/Network.py` selects these classes when `-network=garnet` is used.

4.5 Packet generation, buffering, and downstream eligibility

`GarnetSyntheticTraffic::generatePkt()` → `RubyPort/Sequencer` → `Garnet_standalone cache`
`MessageBuffer` → `NetworkInterface::flitIsSizeMessage()` → source NI VCs → `NetworkLink` →
`Router InputUnit VC` → `SwitchAllocator/Crossbar` → destination NI → directory
`MessageBuffer`

The `generatePkt()` method in `GarnetSyntheticTraffic` chooses a destination and creates a Read, instruction-fetch, or Write request. `RubyPort` and `Sequencer` deliver it to the Garnet standalone cache controller. The controller maps requests to vnet 0, 1, or 2 and enqueues a protocol message. The source `NetworkInterface` allocates an output VC, converts the protocol message into flits, and increments the injected statistics.

During transmission, flits may be stored in the protocol `MessageBuffer`, the source NI's `niOutVcs`, a `NetworkLink` buffer, a router `InputUnit`'s `VirtualChannel::inputBuffer`, a transient crossbar buffer, or the destination NI stall queue. Packets are represented as flits inside Garnet; the tail flit identifies packet completion.

`RoutingUnit` chooses the output direction but does not decide whether the flit may advance. That decision is made primarily by `SwitchAllocator::send_allowed()`. A head flit needs a free downstream output VC; a body or tail flit needs a positive credit, meaning an available slot in the downstream input VC. Ordered vnets must also preserve the arrival order of flits requesting the same output. `OutputUnit` and `OutVcState` maintain credit counts, while reverse `CreditLinks` return freed slots to the upstream router.

At the destination, the `NetworkInterface` places the completed protocol message in the directory input buffer and records receive and latency statistics. The Garnet standalone directory then dequeues and discards it because this protocol is designed to measure network behavior rather than execute a complete cache coherence transaction.

5 Conclusion

The corrected simulation runs for 10000 Ruby cycles and produces packet counts consistent with the configured probabilistic load. The measured average hop count agrees with the theoretical 8×8 mesh distance, and 99.905% of injected packets arrive before termination. Adding explicit units removes ambiguity from Garnet's statistics, while source inspection shows that credit-based flow control and switch allocation govern downstream movement.